

1- What are the columns, what's the meaning of each one, which ones do we keep?

- **Follow.parquet**

Column	Meaning
Did_id	The follower
Subject_id	The account being followed
Created_at	Timestamp of the follow
Rkey	Record key (per follow record)

Keep : did_id, subject_id, created_at

Row count (from metadata): 5,000,000

Time range for 'created_at':

- min: 2023-02-27 15:29:27.657000+00:00

- max: 2029-04-05 23:47:12.533000+00:00 → need to check time errors

- **Likes.parquet**

Column	Meaning
Did_id	User who liked
Subject (dict: did_id (author), collection, rkey)	Details about the liked post
Created_at	When the like happened
Rkey	Record key

Keep : did_id, subject.did_id,, created_at

Row count (from metadata): 20,000,000

Time range for 'created_at':

- min: 2023-01-01 00:26:09.701000+00:00

- max: 2025-04-16 06:57:11.160000+00:00

- **Posts.parquet**

It has many embeds that are useless

Column	Meaning
Did_id	User who posted
Created_at	Timestamp of post creation
Tags	Hashtags mentioned in the post
Labels	Label
Languages	Languages of the post
Reply (dict with root and parent)	Record key

```
{"root": { "did_id": ..., "collection": "...", "rkey": "..." },  
  "parent": { "did_id": ..., "collection": "...", "rkey": "..." }}
```

Keep: did_id, created_at, Label, Languages, reply.root.did_id, reply.parent.did_id
(or just keep reply simply)

Row count (from metadata): 5,000,000

Time range for 'created_at':

- min: 1970-12-31 19:18:00+00:00
- max: 2025-04-16 06:32:32.821000+00:00

- **Reposts.parquet**

Column	Meaning
did_id	User who reposted
subject.did_id	Author of the reposted post
subject.collection	Record collection
subject.rkey	Post ID
created_at	Timestamp of repost
rkey	Internal record key

Keep: did_id, subject.did_id, subject.rkey, created_at

Row count (from metadata): 5,000,000

Time range for 'created_at':

- min: 2023-01-01 00:32:00.119000+00:00
- max: 2025-10-07 17:02:10.682000+00:00

- **List_blocks.parquet (a list is a sort of a “forum” or a “group” on bluesky)**

Column	Meaning
did_id	User who performs the block
subject.did_id	Owner of the list
subject.collection	The list collection
subject.rkey	Record ID of the list
created_at	Timestamp of the block
rkey	Record key of the block

Keep: did_id, subject.did_id, subject.collection, created_at

Row count (from metadata): 4,285,907

Time range for 'created_at':

- min: 2023-09-29 06:23:17.173000+00:00
- max: 2025-04-16 04:37:30.099000+00:00

- **List_blocks.parquet (Mainly the target dataset)**

Column	Meaning
did_id	User who performs the block
subject	Blocked user
created_at	Timestamp of the block
rkey	Internal record key

Keep: did_id, subject.did_id, created_at

Row count (from metadata): 120,088,510

Time range for 'created_at':

- min: 1970-01-01 00:00:00+00:00

- max: 9999-12-31 23:59:59.999000+00:00

- **List_items.parquet**

This is huge (40M rows) and mostly about list membership actions

Column	Meaning
did_id	Owner of the action
list.did_id	Owner of the list
list.collection	Type of list
list.rkey	Record key of the list
subject_id	User added to the list
created_at	When the user was added
rkey	Record key of the action

Keep (optional): did_id, subject_id, created_id, list.didid, list.collection (only if we want to analyze list relationships otherwise, we can ignore this entire dataset)

Row count (from metadata): 43,543,360

Time range for 'created_at':

- min: 0010-01-01 12:16:35.797000+00:00

- max: 2025-04-16 06:47:04.294000+00:00

- **Lists.parquet**

Metadata about lists themselves.

Column	Meaning
did_id	Creator of the list
purpose	Type: moderation list, curated list, etc.
name	Name of the list
description	Description of the list

avatar_uri	Always null
labels	Always null
rkey	Record key

Keep: did_id, purpose, name

Row count (from metadata): 989,009

Time range for 'created_at':

- min: 0010-01-01 12:13:28.015000+00:00

- max: 2025-04-16 01:59:21.718000+00:00

- **Profiles.parquet**

Metadata about lists themselves.

Column	Meaning
did_id	Owner of the profile
Created_at	Creation timestamp
Avatar	Profile picture
Description	Description of the profile
Banner	Banner picture
joined_via_starter_pack	How the user joined bluesky
rkey	Record key
Labels	Labels
Additional_fields	Metada

Keep: did_id, created_at

Row count (from metadata): 32,170,299

Time range for 'created_at':

- min: 0081-11-15 02:48:08.855000+00:00

- max: 2954-08-31 07:36:16.393000+00:00

2- Tasks to be studied:

- First task: block prediction given a user A and a user B predict whether A will block B in the future (a timeline to define)

In this case we should explore the following features:

- Does A follows B (and vice versa)
- Any likes, reposts, replies between A and B ?
- Qualify the type of interaction between them
- See if A has ever been added or removed from lists by B

- ➔ An idea is to perform a binary classification (supervised learning): The prediction outcome will be 1 (if A will block B) and 0 (if not)
Logistic regression or Random Forest can be a good start
- ➔ If the prediction concerns a future edge in a social graph we can consider graph ML models. To see if a user A will perform blocks on a certain community in a graph.
- Second task: Predict whether a specific post will result in someone (from the audience) blocking the author. In this case a possible ML approach could be combining text classification with user history into a classification model
- Third task: social network analysis: Predict whether users from different communities / languages will block each other. We can use:
 - languages
 - list memberships
 - interest clusters (inferred from tags / likes)
- Fourth task : toxicity and conflict detection: Even without post text, you can detect conflict signals: Predict:
 - If two users replying to each other will end up blocking
 - If a chain of reposts signals disagreement
 - If adding someone to a specific list precedes a block
- Optional task: early behavior prediction for new users

3- Needed time window for each task

Task	Ideal Time Window	Why
User A → B Block Prediction	1–3 months	Blocks need long history
Post → Block Reaction	2–4 weeks	Post reactions are short-lived
Community-Level Blocking	1–3 months	Community structure changes slowly
Conflict / Toxicity Detection	2–6 weeks	Replies & conflicts happen quickly
New User Early Behavior	7–14 days	Only early signals matter